

L1 against battery fade

Two hardware hovers, one bare and one carrying 769 g, asked whether the L1 adaptive augmentation can absorb the thrust coefficient a draining 6S pack takes away from it. Unloaded it does, to within a tenth of a newton. Loaded it runs out of clamp.

bags	debug_recording_l1_direct_20260821_191324 / _769g20260821_192638
autopilot	PX4 v1.16 on FMU-v6X (NuttX), logs 258 / 260 – real hardware, not SITL
law	Cai et al. CEP 164 (2025) 106418, hover specialisation – autopilot_drone_geometric_l1_direct_actuation_node
duration	162 s and 161 s armed; DIRECT for essentially all of both

Q1 UNLOADED COMPENSATION

Yes — 94–95 % delivered

u_{L1} climbed 1.16 → 3.07 N against a required 1.33 → 3.23 N. Residual force ≤ 0.17 N; altitude held to 4–10 mm.

Q2 LOADED COMPENSATION

No — the ±10 N clamp binds

Decay 22 % faster, and the demand hit 15.3 N against a 10 N ceiling. The vehicle sagged 0.35 → 0.65 m below its reference.

Q3 DECAY PROFILE

$k_f \propto V^{2.2}$

–6.1 % in 141 s bare, –7.4 % in 139 s loaded. One pack model $V_{oc}(Q)$ – $I \cdot 0.0282 \ \Omega$ fits both to 4.4 mV.

00

What actually flew

Recovered from the logs, because the config on disk is not the config that flew.

Two parameters had to be identified before anything else could be interpreted, and both differ from

config/params_single_drone_geometric_l1_direct_actuation_t650.yaml as it now sits in the working tree.

The allocator's thrust coefficient was 4.679931×10^{-5} , not the +20 % robustness value the file carries. The allocator inverts thrust to rotor speed with its own k_f , so dividing the commanded collective by the modelled $\Sigma \omega^2$ recovers that constant exactly. Over 33 632 samples spanning 20.6 → 38.3 N it came out constant to ten significant figures (std 1.4×10^{-12} , i.e. float32 quantisation):

$$k_{f,ctrl} = (f_b + u_{L1}) / \sum_i (w_{idle} + span \cdot u_i)^2 = 4.679931000 \times 10^{-5} \text{ (yaml on disk: } 5.6159172 \times 10^{-5} \text{)}$$

The L1 sample clock used measured intervals, i.e.

l1adapt_fixed_sample_time_s: 0.0, not the 0.004 the file carries. The low-pass recursion $u \leftarrow \alpha u + (1-\alpha)y$ is exact, so α can be read off consecutive log samples; the implied dt came out 3.44 / 13.05 / 16.62 ms, which are precisely the measured mocap capture intervals and their sum, and nothing like multiples of 4 ms. That is the correct hardware setting — but it means the flown configuration exists only in these logs.

REPRODUCIBILITY

The yaml was last written at 18:19 on flight day, ~55 minutes before the first arm, and it now contains neither flown value. Whatever was edited for the campaign was reverted afterwards. Consider committing a _t650_hardware.yaml so a hardware run can be reproduced from the repo rather than reconstructed from a bag.

01

Does u_{L1} cancel the thrust drop precisely?

Unloaded: yes. The residual is 0.1 N, and it is the estimator's structural floor, not a tuning deficiency.

The DIRECT law has no integral term anywhere. That makes the question

answerable exactly, with no model at all: whatever force the augmentation fails to supply must be carried by the position proportional term, and that term is visible in the logged baseline thrust. Subtracting the gravity feed-forward from f_b leaves precisely the shortfall.

$$u_{L1}^{\text{required}} - u_{L1}^{\text{delivered}} = f_b - m_{\text{model}} \cdot g \cdot R_{\text{z3}} = -(K_p \circ e_p + K_v \circ e_v) \cdot (R \ e_3)$$

No mass, no accelerometer, no thrust-curve assumption enters. On the bare airframe the answer is a residual of 0.09–0.17 N that stays flat while the disturbance it is fighting grows from 1.3 to 3.2 N — so the fraction delivered *improves* through the flight, from 88 % in the first ten seconds to 95 % at the end.

FLIGHT A — BARE T650, HOVER AT 1.5 M							
T [S]	V [V]	P = $K_{\text{EFF}}/K_{\text{CTRL}}$	F _B [N]	U _{L1} [N]	REQUIRED [N]	RESIDUAL [N]	E _z [MM]
24	24.369	0.9618	29.86	1.164	1.330	0.166	−16.
44	24.252	0.9462	29.80	1.711	1.823	0.113	−5.
64	24.155	0.9418	29.81	1.849	1.964	0.115	−6.
84	24.062	0.9364	29.81	2.022	2.139	0.117	−6.
104	23.963	0.9222	29.85	2.481	2.633	0.151	−8.
124	23.878	0.9136	29.83	2.797	2.936	0.140	−8.
144	23.798	0.9054	29.85	3.072	3.227	0.155	−10.

Note what f_b does across that table: nothing. It sits at 29.8 N for the whole flight while the commanded collective climbs from 31.0 to 32.9 N. The baseline law never notices the pack draining — the augmentation absorbs all of it.

Where the last 5 % goes

Of the 0.155 N residual at t = 144 s, only 0.081 N is vertical; the rest is the lateral offset (§04) projected onto the thrust axis through 1.8° of tilt. The vertical part is a steady 2.5–3 % of the disturbance, which is what the piecewise-constant adaptation law is supposed to leave: its fixed point is

$$y_{\text{steady}} = d \cdot e^{A_s \cdot T} \rightarrow \text{under-read of } 1 - e^{-2 \cdot 0.0083} = 1.6 \% \text{ at the measured } 8.3 \text{ ms mean sample}$$

Measured 2.5–3 % against a predicted 1.6 % — the same order, with the excess plausibly from the non-uniform sample cadence (§04). So 95 % is close to this estimator's ceiling. Chasing the last few percent means faster predictor poles A_s (currently −2, the paper uses −65/−80), not position gains.

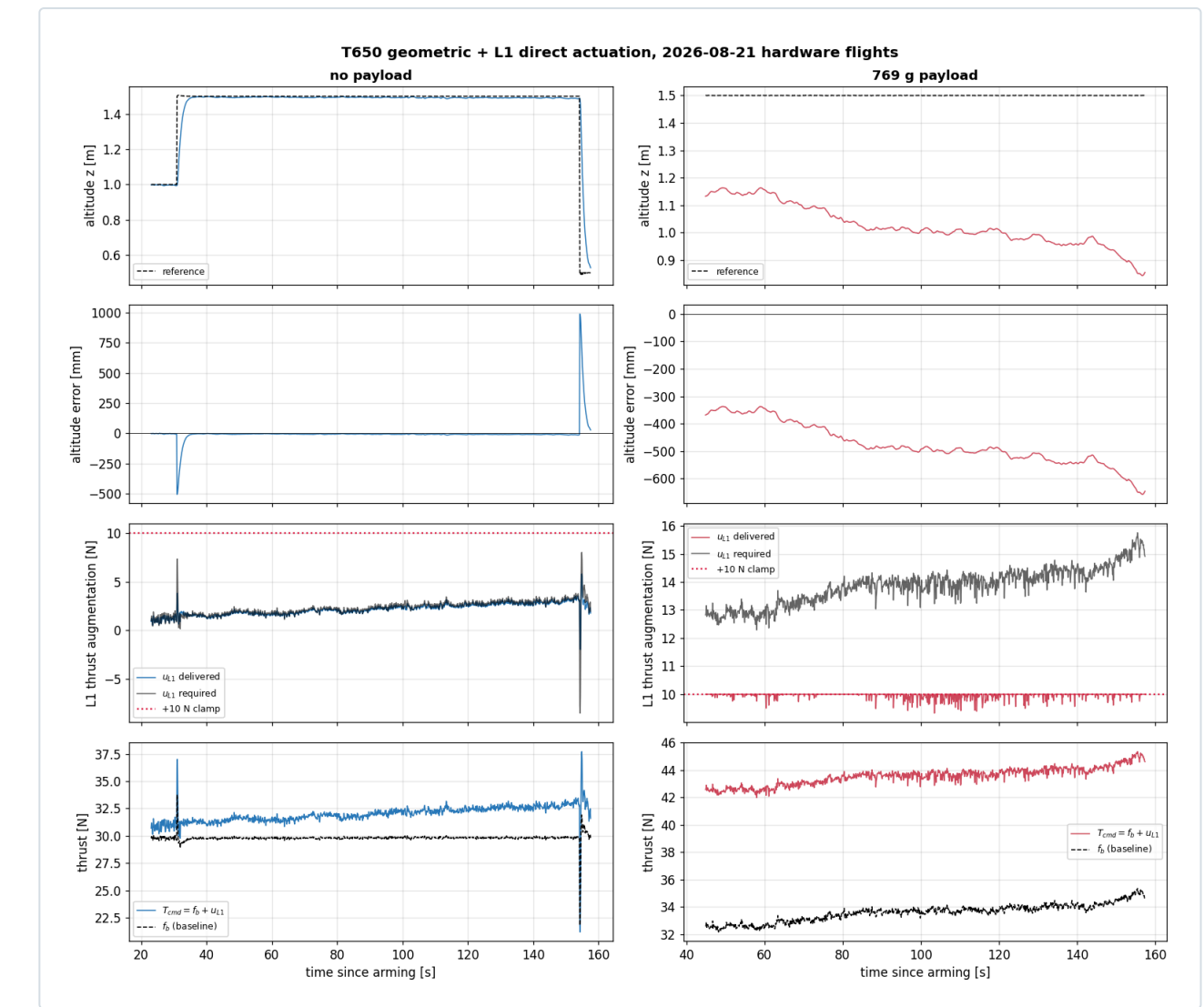


Fig 1 — left column bare, right column loaded. Row 3 is the whole story: unloaded, the delivered and required L1 traces are indistinguishable; loaded, the delivered trace is pinned flat on the 10 N clamp while the requirement walks away from it. Row 4 shows f_b staying at hover weight (bare) or climbing with the position error (loaded) while the commanded collective rises in both.

With a 769 g payload

The decay gets 22 % faster, and the augmentation saturates. The gains are not the problem.

Does the drop rate increase? Yes, by 22 %

Normalised decay over each flight's hover window:

FLIGHT	PACK CURRENT	HOVER THRUST	K _F CHANGE	WINDOW	RATE	ΔV
bare T650	15.1 A	29.8 N	-6.14 %	141 s	-0.0435 %/s	-0.67 V
+ 769 g	20.5 A	38.1 N	-7.37 %	139 s	-0.0529 %/s	-0.79 V

The mechanism is entirely bookkeeping on the pack, not anything aerodynamic: 36 % more current drains charge 36 % faster, and the open-circuit curve converts charge to volts. It is 22 % rather than 36 % only because the loaded flight sat further down the discharge curve, where dV_{oc}/dQ has flattened from -1.5 to -1.0 mV/mAh.

THE MAP ITSELF DOES NOT CHANGE

Both flights fall on *one* k_f(V) curve to within ~2 %, and one pack model $V = V_{oc}(Q) - I \cdot 0.0282 \ \Omega$ reproduces both terminal voltages to 4.4 mV rms. The payload does not make the motors less efficient at a given pack voltage — it just gets you to the low voltages sooner. That is what makes a single simulator profile possible.

Does L1 still compensate? No — it is on its clamp

u_{L1} sat at 9.94–9.99 N for 82.2 % of the DIRECT ticks while the requirement grew from 12.8 to 15.3 N. The vehicle parked wherever the position P-term could make up the difference, and slid further down as the shortfall grew.

The blockage is the *injection*, not the observer. Weighting each update by its own L1 interval — which is what the low-pass actually does — the estimate reached -13.8 N against a fixed point of ≈ -15 N, clipping its own ± 20 N bound on only 22.5 % of the elapsed time. The anti-windup machinery worked as designed (γ_m is deliberately given twice the injection's range, and the predictor is fed the wrench the allocator actually achieved). The augmentation knew what was needed; l1adapt_max_thrust_n would not let it through.

Why the demand is five times larger, in one line

At the adaptation's fixed point the matched estimate is $y_f = (m_{\text{model}}/m_{\text{true}}) \cdot T_{\text{phys}}$ - T_{cmd} , so with $T_{\text{phys}} = \rho \cdot T_{\text{cmd}}$ the augmentation is asked for

u_{L1}^{required} = T_{cmd} · [1 - (m_{model}/m_{true}) · ρ]

— the whole collective, times the fractional error in the composite gain from commanded newtons to predicted acceleration. That gain has two independent multiplicative factors, and the payload flight got hit in both. Evaluated straight from the logged ρ and T_{cmd}, this reproduces the measured requirement to 0.16 N in every row of both flights:

FLIGHT, T	T _{cmd}	m _{model} /m _{true}	ρ	COMPOSITE GAIN	ERROR	DEMAND
bare, 24 s	31.03 N	1.0000	0.9618	0.9618	3.8 %	1.19 N
bare, 144 s	32.92 N	1.0000	0.9054	0.9054	9.5 %	3.11 N
+769 g, 45 s	42.51 N	0.7823	0.8962	0.7011	29.9 %	12.71 N
+769 g, 155 s	44.98 N	0.7823	0.8479	0.6633	33.7 %	15.14 N

The collective grew 1.4× because the vehicle is 28 % heavier, and the fractional error grew 3.5× because a mass the controller was never told about is a 21.8 % gain error on its own. Multiplied, that is a five-fold jump in demand against a ceiling that never moved. Nothing about it is a tuning coincidence — it is arithmetic that was fixed before the vehicle left the ground.

And it cannot recover on its own. Past the clamp, extra error only buys extra position error at K_{p,z} = 8 N/m, and because ρ keeps falling as the pack drains, the gap widens monotonically: 2.9 N of shortfall at t = 45 s becomes 5.3 N at t = 155 s, which is exactly the 0.35 → 0.65 m of sag.

WHERE 10 N CAME FROM

The bound was sized in the 2026-08-21 sim sweep for the deliberately injected +20 % k_f mismatch at *matched mass*, which needs ~5.95 N; 10 N left transient headroom after the earlier AM campaign's 6 N bound railed 93-100 % of DIRECT. It was never sized for an unmodelled 0.84 kg. Note the airframe itself had plenty left — motor commands peaked at 0.72 of 1.0, and the allocator's ceiling is 99.8 N against a 44 N hover. The clamp was the only thing in the way.

FLIGHT B — 769 G PAYLOAD, HOVER REFERENCE 1.5 M								
T [S]	V [V]	P	F _B [N]	U _{L1} [N]	REQUIRED [N]	SHORTFALL [N]	E _Z [MM]	D
45	23.892	0.8962	32.57	9.944	12.820	2.876	-352	

65	23.770	0.8874	32.99	9.957	13.250	3.294	−401
85	23.658	0.8736	33.65	9.976	13.939	3.963	−487
105	23.553	0.8716	33.76	9.969	14.036	4.067	−501
125	23.431	0.8673	33.97	9.981	14.257	4.276	−523
145	23.299	0.8573	34.47	9.983	14.762	4.779	−577
155	23.255	0.8479	34.98	9.991	15.302	5.311	−651

NOT A GAIN-TUNING PROBLEM

The steady-state error is a saturation, and the arithmetic closes exactly: $\text{shortfall} \times \rho \div K_{p,z}$ reproduces the measured e_z at every point in the table. Re-tuning K_p/K_v would only trade altitude error for a stiffer loop — the missing 5 N of thrust would still be missing.

Two thirds of the demand is bookkeeping, not battery

The 12.8–15.3 N splits cleanly, because `vehicle_mass` was left at the bare-airframe 3.033921 kg while the vehicle actually weighed ~3.88 kg:

	AT T = 45 S	AT T = 155 S	
unmodelled mass	8.28 N	8.28 N	<code>vehicle_mass</code> never updated for
thrust-coefficient deficit	4.40 N	6.82 N	the battery fade this campaign
total demand	12.68 N	15.10 N	vs

Set `vehicle_mass` to the loaded mass and the augmentation is asked for only 4.4 → 6.8 N — inside the existing clamp, with the same margin the bare flight enjoyed. I would do both of these before the next loaded flight:

- **Set `vehicle_mass` to the flown mass.** The paper's law puts weight in the baseline feed-forward; leaving it wrong spends the adaptive channel's whole budget on a constant you already know.
- **Raise `l1adapt_max_thrust_n` from 10 to ~15 N.** There is no physical reason for 10: the allocator's ceiling is 99.8 N against a 44 N loaded hover. And this campaign shows the deficit alone reaches 6.8 N by the end of a 2.5-minute flight — a 10 N bound leaves under 50 % margin on a long AM sortie.

WORTH CHECKING ON THE BENCH

The loaded mass implied by the hover balance is 3.878 kg — 75 g more than bare + 769 g. Mounting hardware is the likely answer; the 2026-08-20 stepped-payload campaign already ruled out a downwash penalty on this airframe. It shifts nothing in the decay fits (they are mass-independent), but it does shift the absolute k_f level by ~2 %.

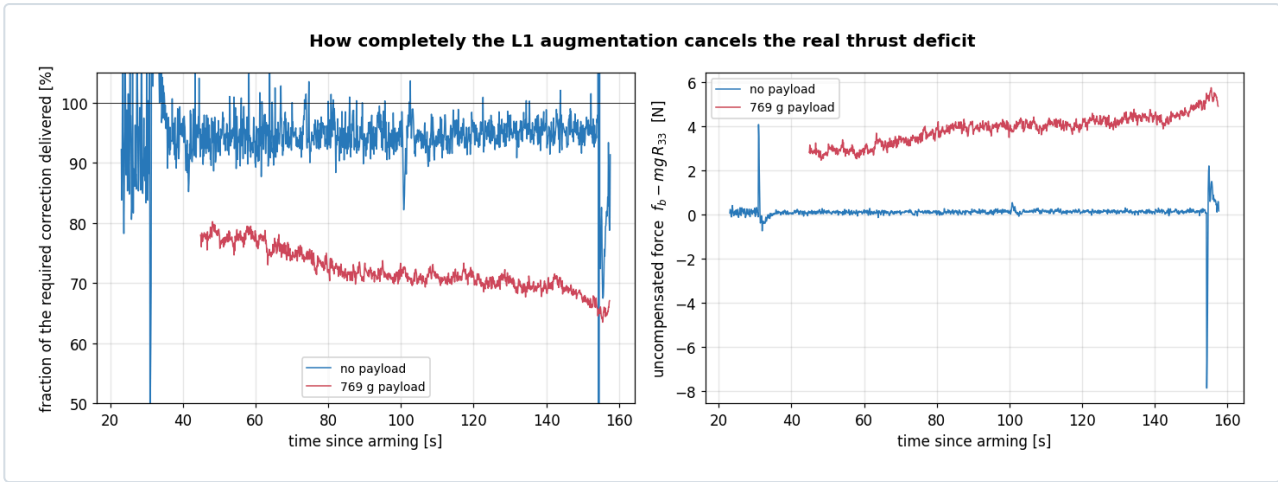


Fig 4 – left, the fraction of the required correction actually delivered; right, the force left for the position loop to carry. Bare (blue) holds a flat ~0.1 N residual all flight. Loaded (red) starts already short and degrades monotonically as the clamp stays fixed and the requirement grows.

03

The fitted decay profile

Fits only — nothing has been wired into the simulator.

The quantity a simulator needs is not the propeller's thrust constant but the coefficient that maps *the rotor speed the allocator believes it commanded* to the thrust actually produced. That lumps the true constant together with the voltage dependence of the ESC throttle → RPM map, which is where nearly all of the fade lives:

$$k_{f,eff}(t) \equiv T_{physical} / \sum_i \omega_{i,model}^2, \quad \omega_{i,model} = 64.0603 + 665.9904 \cdot u_i$$

In hover the vertical balance gives $T_{physical}$ exactly — $k_{f,eff} = m \cdot g / (\cos\theta \cdot \sum \omega^2)$ — so no accelerometer is needed. Total mass enters only as a multiplicative scale, which means **the decay itself is independent of every mass assumption in this report.**

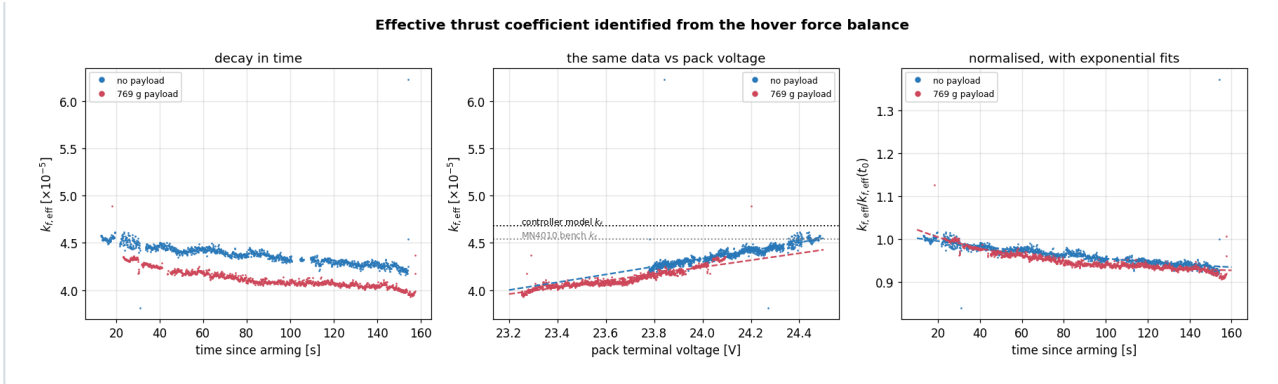


Fig 3 – the middle panel is the useful one: plotted against pack voltage rather than time, the two flights lie on a common curve and simply occupy different stretches of it. The dotted horizontals mark the coefficient the controller was using and the MN4010 bench value.

Level 1 — drop-in time profile

Valid only at the load and state of charge it was measured at. Use it for a quick robustness sweep.

CASE	$K_f(T_0)$	EXPONENTIAL FIT $S(T) = K_f/K_f(T_0)$	RMS	LINEAR
bare, ~15.1 A	4.536959e-05	$0.888190 + 0.121301 \cdot e^{-t/168 \text{ s}}$	1.4 %	1.001188
+769 g, ~20.6 A	4.337508e-05	$0.921808 + 0.121559 \cdot e^{-t/53 \text{ s}}$	0.7 %	0.994365

Level 2 — the physical map

Per-flight exponents came out 2.35 (bare) and 2.05 (loaded); V^2 is the expected law, since a fixed duty cycle scales RPM with terminal voltage and thrust with RPM^2 . Use $n = 2.2$ pooled, or 2.0 if you prefer the clean physics.

$$k_{f,\text{eff}}(V) = 4.30 \times 10^{-5} \cdot (V / 24.0 \text{ V})^{2.2}$$

V = measured pack TERMINAL voltage, under load

Level 3 — portable pack model

This is the one to use if you want the profile to stay right for a different payload, a different hover thrust, or a partly-drained pack, because it separates "how fast the charge goes" from "what the voltage does to the motors". All three pieces were fitted jointly across both flights.

```
Q      : running charge drawn                                [mAh]
I       = 22.1637 * (sum_i omega_i^2 / 1e6) + 0.0145          [A]   rms 0.22 A, both f
V_oc(Q) = 25.015782 - 1.717648e-03*Q + 1.174395e-06*Q^2
          - 6.118624e-10*Q^3                                [V]   6S pack
V       = V_oc(Q) - I * 0.0282                               [V]   rms 4.4 mV, both f
k_f     = 4.30e-05 * (V/24.0)^2.2

Q_next = Q + I*dt/3.6
```

Replaying the two measured hover currents through this loop reproduces the logged terminal voltage to within 5 mV and the identified k_f to within 1 % over both 160-second flights. A reference implementation with a self-test is in `battery_thrust_decay_profile.py` alongside the analysis scripts — nothing imports it.

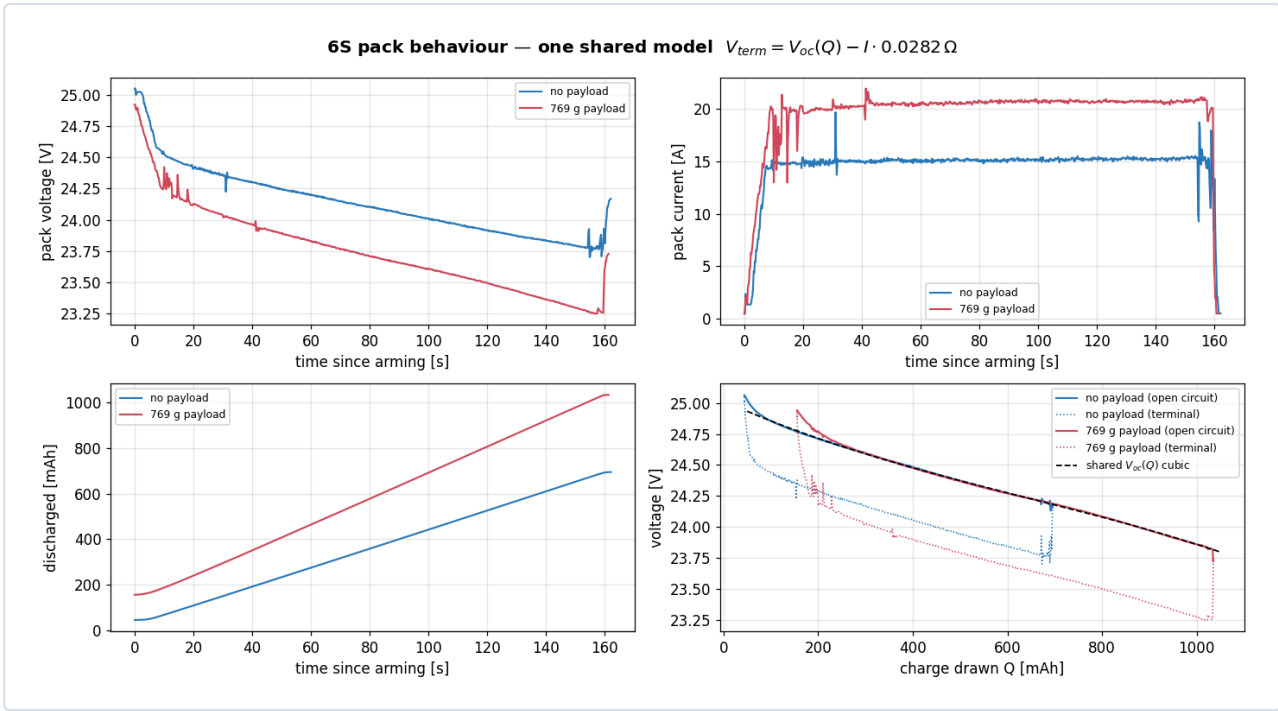


Fig 2 – bottom right is the validation. Terminal voltage (dotted) separates the two flights by their load; add back $I \cdot 0.0282 \Omega$ and both open-circuit traces land on the same cubic (black dashed), across two different flights, two load levels and 50–1050 mAh of charge.

ONE CAVEAT BEFORE YOU TRUST THE ABSOLUTE LEVEL

Every absolute k_f here scales linearly with the assumed mass (3.0339 kg bare, 3.878 kg loaded). The *shape* — decay rate, exponent, pack model — does not. If you want the absolute anchor tight, weigh the airframe as flown; a 1 % mass error is a 1 % k_f error.

Four things found on the way

Not asked for, but each one changes how a log gets read or a parameter gets set.

Reading y_m straight out of the log over-reads by 2.5×

The arithmetic mean of `l1_control_debug[20]` over the bare flight is −5.50 N while the augmentation actually delivered +2.17 N. Nothing is wrong: the piecewise-constant estimate scales as 1/dt, the low-pass weights each update by that same dt, and the two cancel. Sampling the published array at a uniform 250 Hz destroys that weighting. **Index 16 (u_{L1}) is the physically meaningful number; index 20 is not, unless you weight it by its own interval.**

The mocap capture stamps arrive in pairs

Intervals are bimodal — 3.5 ms then 13.1 ms, alternating, averaging the nominal 120 Hz. Since the estimate goes as (predictor error)/dt while velocity noise does not shrink with dt, the short-interval updates are close to pure noise and the raw y_m swings ~4× between consecutive updates. The dt weighting saves the output, but a uniform stamp cadence out of the estimator would give a materially cleaner adaptation, and is probably where the extra 1 % of \$01's residual lives.

A standing pitch moment of 0.5–0.8 N·m

Mean L1 torque about body y is 0.503 N·m bare and 0.805 N·m loaded — 33 % and 54 % of the ±1.5 N·m clamp, never railed. At hover thrust that is 17 mm and 21 mm of CoM x-offset, so the payload moved the centre of mass about 4 mm further forward. Worth folding into the feed-forward rather than leaving to the adaptive channel, for the same reason as the mass.

The hardware allocator was using the simulator's k_f

Measured $k_{f,eff}$ at 24.46 V equals the MN4010 bench constant 4.540431×10^{-5} to −0.08 %. The flown 4.679931×10^{-5} is that bench value times the 1.030724 *simulator* fit factor, so the allocator started the flight 3.0 % optimistic and ended 10.3 % optimistic. The yaml header already says to swap in the bench pair for hardware. Doing so would have removed about half of the correction the augmentation had to find.

PACK V	$K_{F,EFF}$ MEASURED	FLOWN MODEL ERROR	BENCH-VALUE ERROR
24.46	4.5442e−05	+2.99 %	−0.08 %
24.00	4.3329e−05	+8.01 %	+4.79 %
23.79	4.2652e−05	+9.72 %	+6.45 %
23.27	4.2431e−05	+10.30 %	+7.01 %

05

Method

What was assumed, what was measured, and what would move the numbers.

- **Clock alignment.** Each ULog was tied to its bag on the disarm edge, then refined by cross-correlating the motor commands (0.949 correlation, 3.6 ms adjustment). Everything is plotted against seconds since arming.
- **Thrust estimate.** The hover force balance, not the IMU. The specific-force route $T = m \cdot \|a\|$ reads 1.75 % high because airframe vibration rectifies through the norm; the ground-idle calibration cannot remove it because the vibration is thrust-dependent. Using $-a_z$ would fix that, but in a settled hover the balance is exact and needs no accelerometer at all.
- **Hover mask.** $z > 0.7$ m, $|v_z| < 0.06$ m/s, $|v_{xy}| < 0.10$ m/s. 130 s and 128 s of qualifying data respectively.
- **Not checked.** The power-module current calibration — R_{int} and the $I(\Sigma\omega^2)$ fit both inherit whatever scale error it has. $\int I \cdot dt$ does agree with PX4's own `discharged_mah` to 3 %, which is self-consistency, not accuracy.
- **Also present but not analysed.** A persistent lateral offset of −0.21 m (bare) and −0.25 m (loaded) in y, worth about 0.85 N. This is the unmatched channel, which the law deliberately never actuates, so a pure-P outer loop parks at F_{lat}/K_p — the same behaviour the 2026-08-21 sim sweep recorded. Independent of everything above.

Analysis run on python3.10 (numpy 2.2, matplotlib 3.10); bag deserialisation via ROS 2 Humble + px4_msgs.